

ArogyaRakshak: A Multi-Agent AI Decision-Support Platform for Hospital Bill Auditing, Insurance Claim Assistance, Government Scheme Discovery, and Medicine Price Verification

PROJECT SYNOPSIS

BACHELOR OF ENGINEERING

Computer Engineering

SUBMITTED BY

Viraj Jadhao

Anurag Pawar

Sanjali Paygude

Raj Janvekar

Arya Chandak

Under the guidance of

Prof. Deipali V. Gore



Department of Computer Engineering
PES Modern College of Engineering, Pune
Academic Year 2026–27

Contents

1. Title, Domain, Keywords & Team.....	3
2. Background / Introduction.....	4
3. Literature Review	4
4. Objectives.....	6
5. Problem Statement	6
6. Scope	7
7. Brief Description / Proposed System	8
8. System Architecture	9
9. Detailed Module Description	10
10. Algorithmic / Technical Approach.....	12
11. System Workflow / Data Flow.....	13
12. Technical Details.....	14
13. Expected / Achieved Outcomes	15
14. Applications / Use Cases.....	16
15. Limitations	16
16. Future Scope.....	17
17. Project Timeline & Probable Date of Completion	17
18. References	18

List of Figures

Figure 1: ArogyaRakshak System Architecture	9
Figure 2: Kadi Shared-Context Hub & Module Interaction	9
Figure 3: System Workflow / Data Flow	13

List of Tables

Table 1: Review of Relevant Literature and Technologies Used.....	5
Table 2: Module Summary.....	12
Table 3: Technology Stack.....	14
Table 4: Project Timeline	17

1. Title, Domain, Keywords & Team

Title: ArogyaRakshak (आरोग्यरक्षक) — A Multi-Agent AI Decision-Support Platform for Hospital Bill Auditing, Insurance Claim Assistance, Government Scheme Discovery, and Medicine Price Verification

Domain: Artificial Intelligence in Healthcare / HealthTech. Sub-domains: Multi-Agent LLM Systems, Retrieval-Augmented Generation, Optical Character Recognition, Indic Natural Language Processing, Regulatory/Legal-Tech Automation.

Keywords: Healthcare decision support · Multi-agent reasoning · Retrieval-Augmented Generation (RAG) · Large Language Models (LLM) · Optical Character Recognition (OCR) · Cross-lingual / Indic NLP · Named-entity resolution · Health insurance regulation (IRDAI) · Government welfare schemes (PMJAY/MJPJAY) · Vector similarity search (FAISS, pgvector) · FastAPI · Next.js · React Native.

Team: Group of 5 — Viraj Jadhao, Anurag Pawar, Sanjali Paygude, Raj Janvekar, Arya Chandak. Guide: Prof. Deepali V. Gore, Department of Computer Engineering, PES Modern College of Engineering, Pune.

Team Roles

#	Role	Scope
1	Pipeline / Orchestration	Multi-agent reasoning across all five modules and Kadi's entity-resolution pipeline.
2	OCR & Data Layer	Document ingestion, OCR (incl. Devanagari), reference-dataset curation (CGHS, NPPA, PMJAY/MJPJAY).
3	Frontend / UX	Next.js web interface and React Native mobile client; trilingual UI.
4	Infrastructure / Deployment	Docker Compose, PostgreSQL, SSE streaming, CI/CD pipeline.
5	Data, Legal QA & Multilingual Review	Government data structuring, IRDAI/scheme rule accuracy, Hindi/Marathi terminology review.

2. Background / Introduction

Patients discharged from Indian hospitals face a fragmented set of administrative and financial hurdles that persist well beyond treatment: itemised hospital bills that are difficult to verify against any public benchmark; insurance claims that are denied under clauses the patient cannot independently check; welfare schemes such as PMJAY and MJPJAY that remain unclaimed because eligibility is not obvious; and branded medicines bought at full retail price when NPPA-capped generic equivalents exist. Each concern is typically handled, if at all, through a separate manual process, a different office, and a repeated re-explanation of the same facts.

ArogyaRakshak is a Final Year Project that treats these four concerns as instances of a single underlying problem — a patient holding a document (a bill, a prescription, a denial letter, a policy) who needs that document read once and translated into concrete, actionable next steps, in their own language.

The project is built as a production-style monorepo (FastAPI backend, Next.js web client, React Native/Expo mobile client, and six installable Python packages) rather than a notebook or prototype script, so that the system described in this synopsis is the same system that runs, is tested by an automated suite, and is verified in continuous integration.

3. Literature Review

Base Paper: Rahulprasath S. et al., *"AI in Healthcare: Simplifying Medical Reports for Enhanced Patient Comprehension,"* Proc. 2025 Int. Conf. on Emerging Technologies in Computing and Communication (ETCC), Bangalore, India, 2025 (IEEE, DOI: 10.1109/ETCC65847.2025.11108371).

This paper combines OCR-based extraction with a RAG-augmented LLAMA model to summarise medical reports for patients — architecturally the same extraction → RAG → LLM-generation pattern used by BillNyay and SchemeSetu — and was selected as the base paper because it is the most rigorously benchmarked of the candidate papers reviewed (MedMCQA, PubMedQA, MedQA). Its reported limitations — English-only, single-document scope, passive-summary output, and no explicit hallucination-mitigation step — are precisely the four axes (language, linked multi-document context, actionable output, and citation-enforced grounding) along which ArogyaRakshak extends it.

Sr. No.	Project Module	Relevant Paper / Source	Contribution to ArogyaRakshak
1	Kadi / BillNyay / SchemeSetu (base technique)	Rahulprasath et al., “AI in Healthcare”, ETCC 2025	OCR + RAG + fine-tuned LLM pipeline for patient-facing medical document comprehension; base paper for this project (see above).
2	BillNyay (overcharge detection)	“Fraud detection in healthcare billing and claims”, IJSRA 2024; “Dialogue to Detection”, arXiv 2026	NLP-based discrepancy detection between narrative and billed services; supports BillNyay's hybrid rules + RAG + multi-agent design as a defensible pattern.
3	Kadi (Indic entity resolution)	Naamapadam (arXiv/ACL); L3Cube-MahaNER (WILDRE-6, LREC 2022); Aksharantar (Madhani et al., 2022)	Indic NER and open transliteration resources underlying the IndicXlit-based cross-script matching designed into Kadi.
4	BillNyay QA Judge (grounding)	“Reducing Hallucinations in Medical AI Through Citation-Enforced Prompting”, MDPI 2026; HalluGraph, arXiv 2025/26	Citation-enforced grounding and an “Unsupported Sentence Ratio” metric, closely matching the QA Judge agent's design intent.
5	Kadi OCR (Devanagari)	DHCD (UCI); IIIT-HW-Dev (IEEE); CHIPS corpus	Character-, word-, and page-level Devanagari handwriting benchmarks for validating OCR on prescriptions and bills.
6	SchemeSetu (eligibility RAG)	“A Reasoning-Focused Legal Retrieval Benchmark”, arXiv 2025; CBR-RAG	RAG retrieval design for eligibility-style statute reasoning where the query and the relevant rule share little lexical overlap.

Table 1: Review of Relevant Literature and Technologies Used

Across all six rows, no single existing work covers the specific combination attempted here — five linked modules, three languages, and outputs grounded in verifiable Indian government reference data (CGHS, NPPA, PMJAY/MJPJAY, IRDAI) — which is itself part of the case for this project's contribution.

4. Objectives

1. To design a shared document-intake and entity-extraction layer (Kadi) so that a single upload can inform multiple downstream healthcare concerns without repeated data entry.
2. To build a hospital bill auditing module (BillNyay) that benchmarks itemised charges against official CGHS rate schedules and drafts an evidence-backed dispute letter.
3. To automate pre-claim insurance form filing (DaaviSetu) and, separately, post-denial dispute handling (BimaNyay) audited against the IRDAI Master Circular (29 May 2024) with a statutory SLA tracker.
4. To help patients discover and evidence their eligibility for the PMJAY and MJPJAY government welfare schemes (SchemeSetu).
5. To verify branded-medicine pricing against NPPA Schedule-I ceiling rates and surface bioequivalent generic substitutes (DawaCheck).
6. To deliver all of the above trilingually (English, Hindi, Marathi) on both a web and a mobile client, under a strict Bring-Your-Own-Document privacy model with zero persistent storage of raw patient documents.
7. To validate the platform with an automated test suite and continuous-integration guardrails, and to define (and, in a later phase, execute) quantitative evaluation metrics for extraction and mapping accuracy.

5. Problem Statement

Indian patients lack a single, trustworthy, multilingual tool that can read a hospital bill, prescription, insurance denial letter, or policy document once and produce actionable, citably-grounded guidance across hospital overcharge disputes, insurance claim filing and appeals, government scheme eligibility, and medicine price verification — without requiring the patient to re-explain their situation to each system, and without the document itself ever being retained on a server beyond the session in which it is processed.

6. Scope

In Scope (current build)

- Document intake and OCR for hospital bills, prescriptions, denial letters, and policy excerpts (Devanagari and Latin scripts) via web and mobile clients.
- LLM-assisted (Groq-hosted) entity extraction and normalisation into a shared case schema, with a regex/heuristic fallback.
- Five domain modules — BillNyay, DaaviSetu, BimaNyay, SchemeSetu, DawaCheck — each grounded in a specific public Indian government reference dataset.
- Trilingual (English/Hindi/Marathi) user interface and generated legal/administrative documents on both clients.
- Real-time Server-Sent Events progress streaming, PDF generation for appeals and pre-authorisation forms, and an automated test suite with CI guardrails.

Explicitly Out of Scope

- Combined phonetic-transliteration + cross-lingual semantic-embedding entity resolution (IndicXlit + IndicSBERT) in Kadi — designed and documented, targeted for the next development phase, not yet wired into the running pipeline.
- Clinical diagnosis or treatment recommendation of any kind — the platform is strictly informational/administrative, never diagnostic.
- Full production interoperability with the Ayushman Bharat Digital Mission (ABDM) — any such integration would be sandbox-scoped exploratory work, not a committed deliverable.
- Formal execution of the quantitative evaluation benchmarks (PEA, BMA, CFMA/CRMA, WER/CER) — the metrics and target thresholds are defined; execution against a labelled test set is planned for a later evaluation phase.

7. Brief Description / Proposed System

ArogyaRakshak is a monorepo platform consisting of a FastAPI backend (apps/api), a Next.js 16 / React 19 web client (apps/web), a React Native / Expo mobile client (apps/mobile), and six installable Python packages under packages/ — one shared infrastructure package (Kadi) and five domain-reasoning packages (BillNyay, DaaviSetu, BimaNyay, SchemeSetu, DawaCheck).

A patient uploads a document once. Kadi holds the raw file only in a transient in-memory buffer, runs OCR, and extracts structured entities (patient, hospital, diagnosis, procedure, medicine, billing line items) using a Groq-hosted LLM with a deterministic fallback.

With the patient's consent, those entities are then available to whichever domain module is relevant, so a single bill upload can, for example, simultaneously feed a BillNyay overcharge audit and a SchemeSetu eligibility check without a second upload.

Each domain module reasons independently against its own authoritative government reference source and returns a structured result — an audit report, a drafted appeal, a claim package, an eligibility verdict, or a price comparison — while a Server-Sent Events channel streams pipeline progress back to the client in real time.

The system deliberately keeps module responsibilities non-overlapping (a “zero-overlap boundary”, formalised in the project's architecture decision records): BillNyay never touches insurance; DaaviSetu only files claims and hands off denials to BimaNyay; BimaNyay never audits hospital rates; SchemeSetu never touches commercial insurance; DawaCheck never audits hospital procedures.

8. System Architecture

Figure 1 shows the layered architecture: client applications (web and mobile) call a versioned FastAPI gateway, which delegates document processing to the Kadi shared-intelligence layer; Kadi in turn hands normalised entities to whichever of the five domain-reasoning packages is relevant, with PostgreSQL (plus pgvector) and the Groq Cloud LLM API as the shared persistence and inference backends.

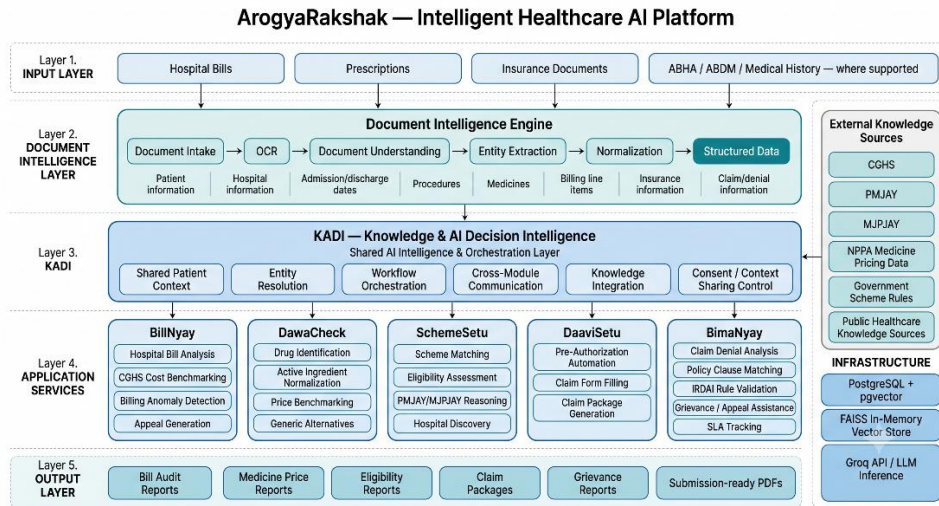


Figure 1: ArogyaRakshak System Architecture

Figure 2 shows the same system redrawn around Kadi as a shared-context hub, emphasising that the five domain modules do not talk to each other directly — every cross-module insight is mediated by Kadi's consent-gated case context, which is what allows one upload to inform multiple modules while keeping each module's internal logic independent and separately testable.

KADI — Knowledge & AI Decision Intelligence

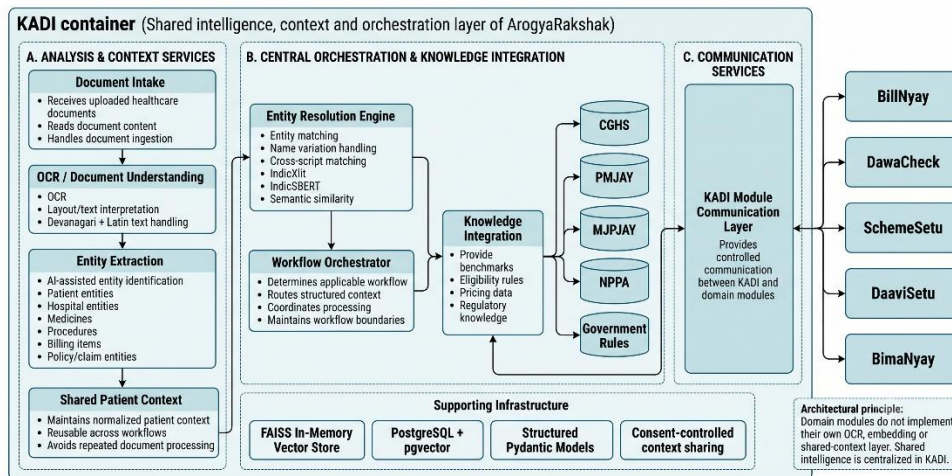


Figure 2: Kadi Shared-Context Hub & Module Interaction

9. Detailed Module Description

Kadi — Shared Intelligence & Context Layer (packages/kadi)

Purpose: pure shared infrastructure — normalises documents and shares extracted context; carries no pricing or legal rules of its own. Inputs: raw uploaded bill/prescription/letter (image or PDF). Processing: OCR via EasyOCR and PyMuPDF (Devanagari and Latin scripts) with a plain-text decoding fallback; entity extraction via a Groq-hosted LLM (openai/gpt-oss-120b) or a regex/heuristic fallback when the LLM is unavailable; candidate blocking via an in-memory FAISS vector index. Outputs: a normalised case-entity JSON persisted against a consent-gated case ID (kadi_cases, kadi_entities). Integration: every domain module reads its working context from Kadi rather than re-parsing the raw document. Note: Kadi's design additionally specifies combined phonetic transliteration (IndicXlit) and cross-lingual semantic embedding (IndicSBERT) matching so that the same entity written in different scripts or languages resolves to one record; this scoring pipeline is documented and planned but not yet implemented in the running code (see Scope and Outcomes).

BillNyay — Hospital Bill Auditing (packages/billnyay)

Purpose: audits itemised hospital bills against CGHS benchmark rates. Inputs: bill line items from Kadi. Processing: a 5-agent chain — Document Auditor (categorises and benchmarks charges against CGHS codes) → Clinical Reviewer (checks medical necessity, flags duplicates) → Regulatory Advisor (flags pricing-notification and consumer-protection violations) → Appeal Drafter (drafts an evidence-backed representation letter) → QA Judge (checks factual grounding before finalising). Technologies: Groq LLM client, a CGHS rate-schedule JSON dataset (packages/billnyay/billnyay/data/cghs_rates.json). Outputs: an overcharge audit report and a dispute PDF. Integration: consumes Kadi context; independent of insurance modules.

DaaviSetu — Claim Application Automation (packages/daavisetu)

Purpose: pre-claim cashless pre-authorisation and reimbursement form filing. Inputs: patient/hospital/diagnosis details from Kadi. Processing: maps case data onto standard insurer templates (e.g. the IRDAI Standard Pre-Authorisation Form, Annexure-B), generating the completed package via ReportLab. Outputs: a downloadable pre-auth/claim PDF and a structured claim-field breakdown. Integration: hands off to BimaNyay if the resulting claim is later denied — DaaviSetu itself never handles denials.

BimaNyay — Claim Denial Appeals & Grievance Tracking (packages/bimanyay)

Purpose: post-denial dispute analysis and statutory appeal drafting. Inputs: a denial/repudiation letter and policy schedule. Processing: a clause auditor checks the repudiation against the IRDAI Master Circular (29 May 2024) — the 5-year moratorium rule, mandatory Claims Review Committee approval, and cashless turnaround-time SLAs — then a drafter produces a 3-tier appeal (Insurer GRO, IRDAI Bima Bharosa complaint, Insurance Ombudsman Form VI), in English, Hindi, or Marathi. Outputs: the tiered appeal drafts plus a timeline tracker computing the 15-day GRO, 15-day Bima Bharosa, and 1-year Ombudsman statutory deadlines. Integration: receives cases from DaaviSetu on denial; independent of BillNyay's CGHS logic.

SchemeSetu — Healthcare Welfare Scheme Advisor (packages/schemesetu)

Purpose: matches patients to PMJAY (national) and MJPJAY (Maharashtra) welfare-scheme eligibility. Inputs: household income, ration-card category, and medical need (from Kadi or a short intake form). Processing: a rule-reasoning agent evaluates eligibility against scheme rules; an offline sentence-embedding fallback (SentenceTransformer, with a deterministic hash-based degradation path when the library is unavailable) supports semantic rule retrieval without depending on network latency. Outputs: an eligibility verdict, confidence indication, and a step-by-step claim guide. Integration: independent of commercial insurance, which is left to DaaviSetu/BimaNyay.

DawaCheck — Medicine Pricing & Generic Alternatives (packages/dawacheck)

Purpose: verifies branded-medicine MRP against NPPA Schedule-I ceiling prices. Inputs: a medicine name/strip photo or prescription text. Processing: normalises the active ingredient and dosage, benchmarks the MRP against the NPPA price-control dataset (with fuzzy alias matching for common brand names), and looks up bioequivalent generic substitutes. Outputs: an overcharge flag and a list of Jan Aushadhi generic alternatives. Integration: independent of hospital procedure billing, which is BillNyay's scope.

Module	Ground-Truth Benchmark	Primary Output
Kadi	N/A — shared infrastructure	Normalised, consent-gated case context
BillNyay	CGHS Rate Schedules	Overcharge audit + dispute letter
DaaviSetu	Standard insurer templates / IRDAI Annexure-B	Pre-auth / reimbursement claim package
BimaNyay	IRDAI Master Circular (29 May 2024)	3-tier appeal drafts + SLA tracker
SchemeSetu	PMJAY & MJPJAY rules	Eligibility verdict + claim guide
DawaCheck	NPPA Schedule-I price orders	Overcharge flag + generic substitutes

Table 2: Module Summary

10. Algorithmic / Technical Approach

1. Document Intake (BYOD): patient uploads a document via web or mobile; the file is held only in a transient in-memory buffer for the duration of the request.
2. Case Initialisation: a Kadi case session is created (POST /api/v1/kadi/cases) with an explicit consent_opt_in flag governing cross-module sharing.
3. OCR: EasyOCR / PyMuPDF extract Devanagari and Latin-script text from the uploaded image or PDF.
4. Entity Extraction & Normalisation: a Groq-hosted LLM (openai/gpt-oss-120b), or a deterministic regex/heuristic fallback, extracts and normalises patient, hospital, diagnosis, procedure, medicine, and billing line-item entities into a shared schema, persisted to kadi_entities.
5. Domain Dispatch: the relevant module (BillNyay, DaaviSetu, BimaNyay, SchemeSetu, or DawaCheck) reads the shared case context and runs its own reasoning chain — a multi-agent LLM chain for BillNyay/BimaNyay, a rule/RAG-based eligibility agent for SchemeSetu, and deterministic benchmarking logic for DawaCheck/DaaviSetu.
6. Real-Time Streaming: a Server-Sent Events channel (GET /api/v1/kadi/cases/{id}/stream) emits stage/progress events (document_received → ocr_parsing → entity_extraction → domain_auditing → completed) throughout processing.

7. Output Assembly: the module returns a structured JSON result and, where applicable, a generated PDF (dispute letter, appeal package, pre-authorisation form) rendered in the requested language.
8. Verification: automated unit tests (per package) and integration tests (apps/api/tests) assert the structure and correctness of each stage; CI guardrails additionally check architectural invariants (BYOD zero-retention, model grounding, naming conventions) on every change.

11. System Workflow / Data Flow

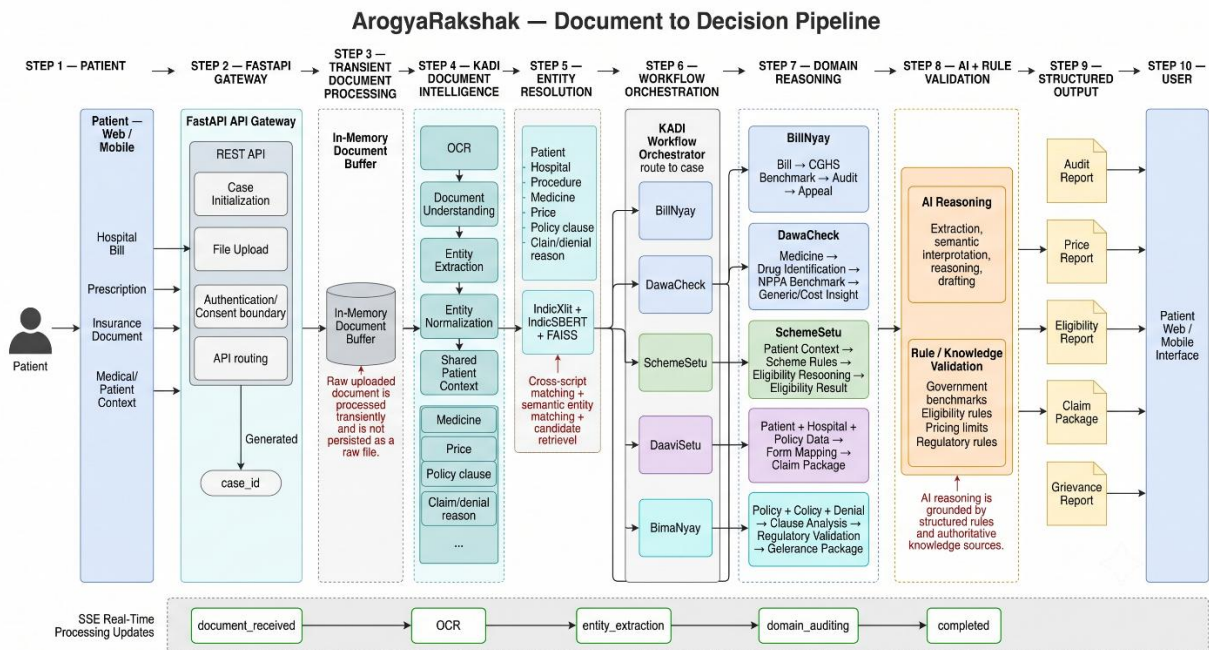


Figure 3: System Workflow / Data Flow

Bring-Your-Own-Document (BYOD) memory lifecycle: the raw uploaded file exists only inside a FastAPI in-memory upload spool for the duration of OCR and extraction, is never written to disk, and is explicitly purged once structured entities have been derived; only de-identified clinical metadata (procedure codes, medicine names, amounts) linked to the ephemeral case ID is persisted to PostgreSQL. This is enforced both by design and by an automated CI guardrail scanning for persistent-document-storage patterns.

12. Technical Details

Category	Details
Platform	Cross-platform development on Windows/Linux/macOS; containerised via Docker Compose (PostgreSQL + pgvector, API, Web).
Languages	Python 3.11 (backend & packages); TypeScript (web and mobile clients).
Backend Framework	FastAPI 0.115 (async), Uvicorn ASGI server, SQLAlchemy 2.0 with asyncpg driver, Pydantic Settings.
AI / ML & NLP	Groq Cloud API (LLM inference, openai/gpt-oss-120b) for multi-agent reasoning and entity extraction; EasyOCR + PyMuPDF for OCR; FAISS for in-memory vector similarity; optional SentenceTransformer-based offline embeddings in SchemeSetu. IndicXlit and IndicSBERT are planned for Kadi's Product Phase 2 and are not yet part of the running pipeline.
Database	PostgreSQL 16 with the pgvector extension for persistent, de-identified entity storage; SQLite in-memory for isolated automated tests.
Frontend (Web)	Next.js 16 (App Router), React 19, TypeScript, vanilla-CSS design tokens; mobile-first (320px–1280px+), WCAG 2.1 AA contrast and touch targets.
Mobile	React Native (New Architecture) with Expo SDK 52, TypeScript; native camera document scanner (expo-camera).
APIs	Versioned REST endpoints under /api/v1 for kadi, billnyay, daavisetu, bimanyay, schemesetu, and dawacheck; Server-Sent Events for live progress; automatic OpenAPI/Swagger documentation.
PDF Generation	ReportLab, for dispute letters and IRDAI Annexure-B pre-authorisation forms.
Development Tools	Git/GitHub with branch protection, GitHub Actions CI (6-stage pipeline), Ruff (Python linting), ESLint (web), pytest, npm test/tsc, Docker Compose.
Hardware / Software Requirements	Development: a standard laptop/desktop (8 GB+ RAM) with Docker Desktop, Python 3.11+, and Node.js 18+/20+; no specialised GPU hardware is required as inference is delegated to the Groq Cloud API.

Table 3: Technology Stack

13. Expected / Achieved Outcomes

The ArogyaRakshak platform is designed to provide an integrated, multilingual healthcare assistance system capable of processing user-provided documents and delivering domain-specific, explainable outputs.

- Unified integration of the five domain modules: **BillNyay, DaaviSetu, BimaNyay, SchemeSetu, and DawaCheck**, supported by the shared **Kadi** processing layer.
- End-to-end document processing involving **OCR, entity extraction, entity resolution, and domain-specific analysis**.
- AI-assisted reasoning and entity extraction using the **Groq Cloud API with openai/gpt-oss-120b**, supported by FAISS-based similarity and optional offline embeddings.
- Multilingual support across **English, Hindi, and Marathi**, including patient-facing outputs and domain-specific document generation.
- Automated generation of domain-specific outputs such as **bill audit reports, insurance appeal documents, claim-related assistance, scheme eligibility information, and medicine price comparisons**.
- Integrated **web and mobile interfaces** for accessible interaction with the platform.
- A modular architecture designed to support **multilingual entity resolution, confidence-based processing, evaluation benchmarks, and future system expansion**.
- A privacy-conscious **Bring-Your-Own-Document** workflow designed around zero-retention processing.

14. Applications / Use Cases

- A patient at a hospital billing desk checking, in the moment, whether room-rent or procedure charges exceed CGHS benchmarks before paying.
- A policyholder whose cashless claim was denied, needing a statutorily-compliant appeal drafted within the 15-day GRO window.
- A low-income family unaware that a hospitalisation is covered under PMJAY or MJPJAY, checking eligibility from a diagnosis and income alone.
- A patient comparing a prescribed branded medicine's MRP against its NPPA ceiling price and nearest generic alternative.
- An NGO, hospital patient-relations desk, or legal-aid clinic supporting multiple patients through the same document-driven workflow in their preferred language.

15. Limitations

- Entity resolution across scripts/languages currently relies on basic string similarity and FAISS blocking; the more accurate combined IndicXlit + IndicSBERT scoring is not yet integrated, so cross-lingual matching accuracy has not been formally measured.
- LLM reasoning depends on the Groq Cloud API; correctness of generated appeals/audits has not yet been benchmarked against a labelled ground-truth set, only unit/integration-tested for structural correctness.
- OCR accuracy on handwritten Devanagari prescriptions has not yet been validated against a dedicated handwriting benchmark.
- The platform is strictly informational/administrative and must not be used, and is not designed, for clinical diagnosis.
- Coverage is limited to the specific government schemes and rate schedules currently encoded (CGHS, NPPA Schedule-I, PMJAY, MJPJAY, the IRDAI 2024 Master Circular); other state schemes or insurers' proprietary rules are out of scope.

16. Future Scope

- Mobile push notifications for statutory GRO/Bima Bharosa SLA deadlines.
- Geolocation of nearby Jan Aushadhi Kendras and PMJAY/MJPJAY empanelled hospitals.
- Sandbox-scoped exploration of Ayushman Bharat Digital Mission (ABDM) interoperability for consented discharge-summary retrieval — not a committed production integration.
- Longer-horizon, exploratory directions noted in the team's internal roadmap (e.g. voice-based intake, AR-assisted bill scanning) remain conceptual and are not scheduled deliverables for this submission.

17. Project Timeline & Probable Date of Completion

The project follows a phased development roadmap, with each phase focusing on progressively expanding the system's capabilities, integration, multilingual support, evaluation, and final deployment.

Phase	Description
Phase 1	Monorepo scaffolding, CI/CD pipeline, Kadi OCR & entity extraction, BillNyay/DaaviSetu/SchemeSetu/DawaCheck module logic.
Phase 2	BimaNyay module (IRDAI appeals & SLA tracker), full mobile app (Expo), end-to-end web + mobile wiring to real endpoints, hardened CI/CD.
Phase 3	Integrate IndicXlit + IndicSBERT cross-lingual entity resolution in Kadi; confidence-scored cross-module auto-triggering.
Phase 4	Devanagari OCR validation on real/synthetic documents; multilingual terminology QA; execution of PEA/BMA/CFMA/CRMA/WER evaluation benchmarks.
Phase 5	Final deployment, end-to-end demo hardening, SPPU Blackbook documentation and submission.

Table 4: Project Timeline

18. References

- [1] Rahulprasath S., Pranav Harshan S., Kabilash P. V., Lakshithraj A., and Sreemathy J., “AI in Healthcare: Simplifying Medical Reports for Enhanced Patient Comprehension,” in Proc. 2025 Int. Conf. on Emerging Technologies in Computing and Communication (ETCC), Bangalore, India, 2025, doi: 10.1109/ETCC65847.2025.11108371.
- [2] Mariammal, Swetha K., and Samuel Jimrys S., “Medical Report Simplification System using NLP-based Extraction and AI-Driven Explanation,” Proc. ICICV 2025.
- [3] Singh, Mehrotra, and Rosaline, “Healthcare Record Optimization using Natural Language Processing and Computer Vision (MedScribeAI),” Proc. GCWCN 2025.
- [4] K. Madhani et al., “Aksharantar: Towards Building Open Transliteration Tools for the Next Billion Users,” arXiv preprint, 2022.
- [5] R. Litake et al., “L3Cube-MahaNER: A Marathi Named Entity Recognition Dataset and BERT Models,” in Proc. WILDRE-6 Workshop, LREC 2022.
- [6] “Naamapadam: A Large-Scale Named Entity Annotated Data for Indic Languages,” arXiv / ACL.
- [7] “Reducing Hallucinations in Medical AI Through Citation-Enforced Prompting in RAG Systems,” MDPI, 2026.
- [8] “HalluGraph: Auditable Hallucination Detection for Legal RAG,” arXiv, 2025/2026.
- [9] “Fraud detection in healthcare billing and claims,” International Journal of Science and Research Archive, 2024.
- [10] “Dialogue to Detection: A Multimodal Hybrid NLP Pipeline for Insurance Fraud Detection,” arXiv, 2026.
- [11] “A Reasoning-Focused Legal Retrieval Benchmark,” arXiv, 2025.
- [12] “CBR-RAG: Case-Based Reasoning for RAG in Legal Question Answering,” arXiv.
- [13] Insurance Regulatory and Development Authority of India, “Master Circular on Health Insurance Business,” IRDAI, 29 May 2024.
- [1] Central Government Health Scheme (CGHS), Ministry of Health and Family Welfare, Government of India — CGHS Rate Schedules.
- [2] National Pharmaceutical Pricing Authority (NPPA), “Schedule-I Ceiling Price Orders under the Drugs (Prices Control) Order, 2013.”
- [3] National Health Authority, Government of India — Ayushman Bharat Pradhan Mantri Jan Arogya Yojana (PMJAY) Guidelines.
- [4] Government of Maharashtra — Mahatma Jyotirao Phule Jan Arogya Yojana (MJPJAY) Guidelines.